Guia Prático & Atividade de Laboratório — Aula 05

Qualidade em APIs: Tratamento Previsível de Erros (ApiError), Testes Unitários de Serviço (JUnit 5 + Mockito) e Observabilidade com Spring Boot Actuator

Módulo: Java Backend Developer	**Tecnologias:** Java 21, Spring Boot 4.1.x, JUnit 5, Mockito, Actuator
Duração Prevista: 45-60 minutos	**Checkpoint Git:** checkpoint/aula-05-quality

1. Resumo Teórico & Dicas de Arquitetura (Guia de Estudo)

Nas Aulas 01 a 04, evoluímos a aplicação MyMovies de um protótipo em memória para uma API REST completa com persistência relacional H2/JPA e integração externa com o TMDB via RestClient. Na Aula 05, elevamos o nível de maturidade da aplicação para padrões corporativos de **Qualidade, Tratamento de Exceções, Testes Automatizados e Observabilidade**.

1.1. Contrato de Erro Previsível (ApiError) & Sanitização Pública

Quando uma requisição HTTP falha (ex.: recurso não encontrado ou dados inválidos), o cliente REST não deve receber rastros de pilha internos (stack traces) nem respostas genéricas HTML de erro 500. A especificação da nossa API exige a devolução de um payload JSON RFC 7807 sanitizado e padronizado:

- **timestamp (Instant):** Data e hora exatas da ocorrência no formato ISO-8601 UTC.
- **status (int):** Código de status HTTP numérico correspondente (ex.: 404, 400, 500).
- **code (String):** Identificador funcional estável do tipo de erro (ex.: 'MOVIE_NOT_FOUND', 'VALIDATION_ERROR', 'INTERNAL_ERROR').
- **message (String):** Mensagem pública clara e informativa sobre a falha.
- **path (String):** URI do recurso HTTP que originou a requisição (ex.: '/api/movies/99').

1.2. Interceção Global de Exceções (@RestControllerAdvice e @ExceptionHandler)

Em vez de espalhar blocos try-catch defensivos nos Controllers, a arquitetura Spring desacopla o tratamento de erros utilizando a anotação **@RestControllerAdvice**. Esta classe funciona como um intercetor global na camada Web que captura exceções lançadas por qualquer Controller ou Serviço e as traduz nos devidos DTOs de erro:

- **Exceções de Domínio (ex.: MovieNotFoundException):** Lançadas pela camada de negócio e traduzidas para HTTP 404 Not Found com código 'MOVIE_NOT_FOUND'.
- **Erros de Validação (MethodArgumentNotValidException):** Lançados automaticamente pelo Bean Validation (@Valid) e traduzidos para HTTP 400 Bad Request com código 'VALIDATION_ERROR'.
- **Falhas Inesperadas (Exception.class):** Erros genéricos de infraestrutura traduzidos para HTTP 500 Internal Server Error, omitindo detalhes sensíveis do servidor.

1.3. Níveis de Logging e Diagnóstico Estruturado (SLF4J)

O registo de logs via SLF4J (Simple Logging Facade for Java) com Logback permite monitorizar o comportamento da aplicação em ambiente de desenvolvimento e produção. Utilizamos o LoggerFactory para registar eventos com níveis semânticos apropriados (DEBUG para rastreio detalhado e ERROR para exceções operacionais).

1.4. Estratégia Prática de Testes: Foco na Camada de Serviço (MovieServiceTest)

> 🎯 **DECISÃO ARQUITETURAL & PEDAGÓGICA DA AULA 05:** Para otimizar a carga cognitiva e o tempo de laboratório, a prática obrigatória foca-se na escrita de testes unitários limpos para a camada de serviço (MovieServiceTest) com JUnit 5 e Mockito. Testes de serviço rodam em milissegundos sem necessidade de subir o servidor Spring. A validação dos contratos HTTP do @RestControllerAdvice é efetuada visualmente no Bruno Client e apresentada como Desafio Opcional via MockMvc.

A suíte de testes unitários do **MovieServiceTest** valida os dois pilares da regra de negócio de pesquisa de filmes:

- **Cenário de Sucesso (Happy Path):** Garante que ao pesquisar por um ID existente, o serviço interage com o repositório mockado e devolve o DTO MovieResponse correto.
- **Cenário de Ausência (Error Path):** Garante que ao pesquisar por um ID inexistente, o serviço dispara a exceção de domínio MovieNotFoundException utilizando a asserção assertThrows().

1.5. Observabilidade Operacional com Spring Boot Actuator

O módulo **spring-boot-starter-actuator** fornece endpoints de produção para inspecionar a saúde da aplicação (/actuator/health) e métricas de desempenho (/actuator/metrics), tais como o volume de requisições HTTP e o uso de memória Heap da JVM.

2. Baseline Git & Estrutura Oficial de Pacotes

Garanta que a sua branch da Aula 05 está criada a partir do progresso local da Aula 04:

```
# 1. Atualizar todas as tags do repositório remoto
 git fetch --tags

# 2. Mudar para a tag oficial do checkpoint da Aula 04
 git checkout checkpoint/aula-04-tmdb

# 3. Criar a nova branch de trabalho para a Aula 05
 git checkout -b feature/aula-05-qualidade-testes
```

A estrutura de pacotes atualizada sob **pt.rumos.mymovies** incluirá os novos pacotes de exceção, tratamento de erros e testes:

```
pt.rumos.mymovies/
├── config/
│   └── MongoMemoryConfig.java      # Configuração MongoDB em memória
├── exception/                      # [NOVO] Exceções de domínio
│   └── MovieNotFoundException.java
├── web/                            # [NOVO] Pacote web & tratamento de erros
│   └── error/
│       ├── ApiError.java           # DTO Record de erro padronizado
│       └── ApiExceptionHandler.java# @RestControllerAdvice global
├── client/
│   └── TmdbClient.java
├── dto/
│   ├── MovieRequest.java
│   ├── MovieResponse.java
│   └── tmdb/
├── model/
│   └── Movie.java
├── repository/
│   └── MovieRepository.java
├── service/
│   ├── MovieService.java          # Atualizado com .orElseThrow()
│   └── TmdbService.java
└── controller/
    ├── MovieController.java
    └── TmdbMovieController.java

src/test/java/pt/rums/mymovies/
└── service/
    └── MovieServiceTest.java       # [NOVO - CORE PRÁTICO DO ALUNO] Teste Unitário
JUnit 5 + Mockito
```

3. Roteiro Guiado da Atividade Prática (Passo a Passo)

Etapa 1: Criar a Exceção de Domínio (MovieNotFoundException) e Refatorar o MovieService

1. Crie o pacote `pt.rumos.mymovies.exception` e adicione a classe `MovieNotFoundException` estendendo `RuntimeException`.

2. No construtor que recebe `Long id`, passe a mensagem `"Movie not found: " + id` para o `super(message)`.

3. Na classe `MovieService`, atualize o método `findById(Long id)` de modo a usar `.orElseThrow(() -> new MovieNotFoundException(id))` sobre o `Optional` retornado pelo `MovieRepository`.

Etapa 2: Implementar o DTO de Erro (ApiError) e o Intercetor Global (ApiExceptionHandler)

1. Crie o pacote `pt.rumos.mymovies.web.error` e defina o Java Record `ApiError` com os cinco componentes do contrato (`timestamp`, `status`, `code`, `message`, `path`).

2. Crie a classe `ApiExceptionHandler` anotada com `@RestControllerAdvice`.

3. Implemente o método `handleMovieNotFound` anotado com `@ExceptionHandler(MovieNotFoundException.class)` e `@ResponseStatus(HttpStatus.NOT_FOUND)`, construindo e retornando o `ApiError` com o código `'MOVIE_NOT_FOUND'`.

4. Adicione o manipulador para `@ExceptionHandler(MethodArgumentNotValidException.class)` com `@ResponseStatus(HttpStatus.BAD_REQUEST)` e código `'VALIDATION_ERROR'`.

5. Adicione o manipulador genérico para `@ExceptionHandler(Exception.class)` com `@ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)` e código `'INTERNAL_ERROR'`.

Etapa 3: Adicionar Registos de Log Estruturados com SLF4J

1. Na classe `ApiExceptionHandler`, adicione um atributo de Logger: `private static final Logger log = LoggerFactory.getLogger(ApiExceptionHandler.class);`.

2. No método de tratamento de erro genérico (500), adicione um registo de log: `log.error("event=API_UNEXPECTED path={} type={}", request.getRequestURI(), ex.getClass().getSimpleName(), ex);`.

3. No `application.properties`, configure o nível de log do projeto para DEBUG: `logging.level.pt.rumos.mymovies=DEBUG`.

Etapa 4: Desenvolver os Testes Unitários da Camada de Serviço (MovieServiceTest)

Na pasta **src/test/java/pt/rumos/mymovies/service**, crie a classe **MovieServiceTest** utilizando JUnit 5 e Mockito sem carregar a infraestrutura do Spring:

1. Anote a classe de teste com `@ExtendWith(MockitoExtension.class)`.

2. Declare o mock do repositório: `@Mock private MovieRepository movieRepository;`.

3. Declare o serviço sob teste: `@InjectMocks private MovieService movieService;`.

4. Implemente o teste de sucesso `findById_WhenMovieExists_ShouldReturnMovieResponse()`: configure `when(movieRepository.findById(1L)).thenReturn(Optional.of(movie))` e valide os campos com `assertEquals()`.

5. Implemente o teste de ausência `findById_WhenMovieDoesNotExist_ShouldThrowMovieNotFoundException()`: configure `when(movieRepository.findById(99L)).thenReturn(Optional.empty())` e valide a exceção com `assertThrows(MovieNotFoundException.class, () -> movieService.findById(99L))`.

4. DESAFIO EXTRA & EXTENSÕES (MockMvc, Bruno & Actuator)

Atividade Extra A (Opcional): Testes de Contrato Web com MockMvc (MovieControllerTest)

Para os alunos que concluírem a Etapa 4 antecipadamente, implemente o teste de contrato na camada web em **src/test/java/pt/rumos/mymovies/controller/MovieControllerTest.java** utilizando

`MockMvcBuilders.standaloneSetup()` para validar o retorno HTTP 404 e o código 'MOVIE_NOT_FOUND' do `ApiExceptionHandler`.

Atividade Extra B: Validação do Contrato de Erro no Bruno Client

1. Execute a aplicação Spring Boot.

2. No Bruno Client, faça um `GET http://localhost:8080/api/movies/99` (ID inexistente). Confirme a resposta HTTP 404 Not Found e a presença do JSON ApiError com `"code": "MOVIE_NOT_FOUND"`.

3. Faça um `POST http://localhost:8080/api/movies` enviando um JSON com título em branco `{"title": ""}`. Confirme a resposta HTTP 400 Bad Request com `"code": "VALIDATION_ERROR"`.

Atividade Extra C: Spring Boot Actuator & Observabilidade Operacional

1. Adicione a dependência `spring-boot-starter-actuator` no `pom.xml`.

2. No `application.properties`, exponha os endpoints: `management.endpoints.web.exposure.include=health,metrics`.

3. No Bruno, faça requisições a `http://localhost:8080/actuator/health` (status UP) e `http://localhost:8080/actuator/metrics/http.server.requests`.